

DevCore AI — Deep Technical Implementation Guide

Production Multi-Agent Architecture, File Breakdown, 35 Component Deep-Dives & Viva Exam Guide

1. Executive Summary & Project Overview

DevCore AI (Autonomous Multi-Agent Software Development OS) is an advanced, enterprise-grade AI software engineering workspace powered by **LangGraph (StateGraph)**, **Streamlit SPA**, **Groq Cloud LLM API (llama-3.3-70b-versatile)**, **Local Laptop Ollama (qwen3.5:9b)**, **SQLite3 WAL Database**, **NumPy Semantic Vector RAG**, **Interactive Mermaid.js SVG Engines**, and **ReportLab PDF Exporters**.

The platform enables software engineering teams to define project requirements, upload SRS/PDF specifications, execute an autonomous **13-Agent Workflow Graph** (CEO, Business Analyst, Project Manager, Lead Architect, UI/UX Lead, Frontend, Backend, Database, Security, DevOps, QA, Documentation, Reviewer), render interactive pan/zoom architecture diagrams, synthesize complete 5-to-10 file codebase implementations inside the Code Generator Studio, and compile single-click PDF technical deliverables.

2. Project Folder & File Breakdown

```
project 2 multi ai/
■■■■ app.py                # Main entry point & Streamlit SPA layout router
■■■■ config.py             # System-wide configuration & model defaults
■■■■ database/            # SQLite Transactional Layer
■   ■■■■ connection.py    # Thread-safe connection pool (WAL mode)
■   ■■■■ schema.py        # DDL schema initialization & default settings
■■■■ agents/              # Specialist AI Agent Layer
■   ■■■■ base.py          # Core agent node executor & SQLite persistence
■   ■■■■ prompts.py       # Master system prompts for 13 AI personas
■■■■ workflow/            # Multi-Agent State Machine Layer
■   ■■■■ state.py         # TypedDict definition of shared ProjectState
■   ■■■■ graph.py         # LangGraph StateGraph builder & execution pipeline
■■■■ pages/               # Page View Modules
■   ■■■■ dashboard.py     # Real-time metrics grid & global project telemetry
■   ■■■■ new_project.py   # 4-Step Project Wizard & RAG file ingestion
■   ■■■■ projects.py      # Spatial Workspace Explorer & Mermaid SVG Engine
■   ■■■■ ai_team.py       # Interactive 13-Agent organizational directory
■   ■■■■ chat.py          # Multi-agent persistent consultation chatbot
■   ■■■■ settings.py      # Engine provider selector & Groq API key manager
■■■■ components/          # Reusable UI & Feature Modules
■   ■■■■ implementation_studio.py # Code Generator Studio (80% Code, 20% Explanation)
■■■■ utils/               # Core System Engines & Utilities
■   ■■■■ config.py        # Settings sync & provider resolver
■   ■■■■ ollama_client.py  # Central LLM Router (Groq API + Local Ollama)
■   ■■■■ implementation_engine.py # Token-budgeted multi-file code synthesizer
■   ■■■■ blueprint_context.py # Context aggregator for 13-agent runs
■   ■■■■ pdf_generator.py  # ReportLab PDF technical manual compiler
```

3. High-Level System Architecture

DevCore AI operates on a dual-engine architecture:

1. **Multi-Agent Blueprint Subsystem (LangGraph)**: Executes 13 specialized engineering nodes sequentially (CEO -> BA -> PM -> Architect -> UI/UX -> Frontend -> Backend -> Database -> Security -> DevOps -> QA -> Documentation -> Reviewer) with shared state accumulation.
2. **Code-First Implementation Engine**: Allocates an extended 8,192 token budget to synthesize production-ready, runnable multi-file source code packages for chosen architecture modules.

4. Diagrammatic Specifications

4.1 Project Architecture Diagram

```
graph TD
    User([User]) --> UI[Streamlit UI - app.py]
    UI --> Wizard[New Project Wizard - pages/new_project.py]
    Wizard --> DB[(SQLite3 WAL DB - ai_software_company.db)]
    Wizard --> RAG[NumPy Vector Store - pages/new_project.py]

    Wizard --> LangGraph[LangGraph StateGraph - workflow/graph.py]

    subgraph Agent_Workforce [13-Agent Engineering Workforce]
        LangGraph --> CEO[CEO Node - agents/base.py]
        CEO --> BA[Business Analyst Node]
        BA --> PM[Project Manager Node]
        PM --> Architect[Lead Architect Node]
        Architect --> UIUX[UI/UX Lead Node]
        UIUX --> Frontend[Frontend Lead Node]
        Frontend --> Backend[Backend Lead Node]
        Backend --> Database[Database Specialist Node]
        Database --> Security[Security Lead Node]
        Security --> DevOps[DevOps Specialist Node]
        DevOps --> QA[QA Lead Node]
        QA --> Docs[Documentation Specialist Node]
        Docs --> Reviewer[Reviewer Node]
    end

    Agent_Workforce --> Router[Dynamic LLM Router - utils/ollama_client.py]
    Router --> Groq[Groq Cloud API - llama-3.3-70b-versatile]
    Router --> Ollama[Local Laptop Ollama - qwen3.5:9b]

    Agent_Workforce --> DB
    DB --> Explorer[Workspace Explorer - pages/projects.py]
    Explorer --> Studio[Implementation Code Studio - components/implementation_studio.py]
    Explorer --> PDFExp[ReportLab PDF Compiler - utils/pdf_generator.py]
```

5. Detailed Feature & Function Deep-Dives (All 35 System Components)

Feature 1: Main Streamlit SPA & Page Navigation Engine

- **Purpose:** Establishes global page configuration, injects custom dark glassmorphism CSS, and routes application pages without browser refreshes.

- **Location:** [app.py](#) -> [Lines 1–80](#)

- **Code Snippet:**

```
st.set_page_config(page_title="DevCore AI", layout="wide")
st.html(get_css())
```

- **Input:** User top navbar clicks or session state transitions.

- **Output:** Rendered active page view (Dashboard, New Project, Projects, AI Team, Chat, Settings).

- **Step-by-Step Logic:**

1. Executes `st.set_page_config` as first call.
2. Injects dark glassmorphism CSS styling tokens.
3. Checks `st.session_state['current_page']`.
4. Delegates rendering to target page function.

- **Why Needed:** Guarantees visual consistency, dark glassmorphic aesthetics, and prevents `KeyError` crashes.

- **Library/Class Used:** `streamlit.set_page_config`, `streamlit.html`

- **Possible Viva Question:** "Why must `st.set_page_config` be the very first Streamlit call in `app.py`?"

- **Ideal Viva Answer:** "Streamlit requires layout and title configurations to be initialized before any DOM elements or session variables are instantiated. Calling visual widgets prior raises `StreamlitSetPageConfigMustBeFirstCommandError`."

Feature 2: Thread-Safe SQLite3 Connection Pool & WAL Mode

- **Purpose:** Manages multi-threaded database access for storing project metadata, agent runs, settings, and chat history.

- **Location:** [database/connection.py](#) -> `get_db_connection()` (Line 12)

- **Code Snippet:**

```
conn = sqlite3.connect('ai_software_company.db', check_same_thread=False, timeout=30.0)
conn.execute('PRAGMA journal_mode=WAL;')
```

- **Input:** SQL queries and parameters.

- **Output:** `sqlite3.Row` objects with dictionary-style column access.

- **Step-by-Step Logic:**

1. Opens SQLite database file with `check_same_thread=False`.
2. Enables `PRAGMA journal_mode=WAL;` for concurrent read/write support.
3. Sets `row_factory = sqlite3.Row`.

- **Why Needed:** Eliminates 'database is locked' errors during parallel background agent execution.

- **Library/Class Used:** `sqlite3`

- **Possible Viva Question:** "Why is Write-Ahead Logging (WAL mode) critical for SQLite in Streamlit?"

- **Ideal Viva Answer:** "By default, SQLite locks the entire database file during writes. WAL mode permits multiple readers to read concurrently while a background thread writes to a separate log file."

Feature 3: 13-Agent Workflow Graph (LangGraph State Machine)

- **Purpose:** Orchestrates 13 senior engineering personas in a deterministic state graph sequence.
- **Location:** [workflow/graph.py](#) -> `build_workflow_graph()` (Line 41)

- **Code Snippet:**

```
workflow = StateGraph(ProjectState)
for role in roles: workflow.add_node(role, create_agent_node(role))
workflow.set_entry_point('ceo')
```

- **Input:** ProjectState dictionary.
- **Output:** Compiled StateGraph execution engine.
- **Step-by-Step Logic:**
 1. Instantiates StateGraph(ProjectState).
 2. Registers 13 agent role nodes.
 3. Defines linear dependency edges from CEO down to Reviewer.
- **Why Needed:** Provides structured state management, cycle safety, and shared context propagation across engineering roles.
- **Library/Class Used:** `langgraph.graph.StateGraph`
- **Possible Viva Question:** **"What is the primary advantage of LangGraph over traditional sequential loops?"**
- **Ideal Viva Answer:** *"LangGraph enforces a typed shared state contract (ProjectState), handles automatic state propagation between nodes, and allows inspectable graph execution."*

Feature 4: Dynamic LLM Router & Groq API / Local Ollama Engine

- **Purpose:** Routes LLM requests to Groq Cloud API or Local Laptop Ollama based on provider selection and key availability.

- **Location:** [utils/ollama_client.py](#) -> [query_ollama\(\)](#) (Line 147)

- **Code Snippet:**

```
if exec_provider == 'groq' or bool(get_any_groq_key()):  
    return query_groq_api_fallback(system_prompt, user_prompt, agent_role=agent_role)
```

- **Input:** system_prompt, user_prompt, agent_role.

- **Output:** Sanitized string output from Groq Cloud API or Local Ollama.

- **Step-by-Step Logic:**

1. Resolves execution_provider setting.
2. Fetches role-specific Groq API key (blueprint, studio, chatbot, consultation).
3. Streams live output from Groq Cloud API (llama-3.3-70b-versatile).
4. Falls back to local Ollama or emergency technical generator if API fails.

- **Why Needed:** Guarantees high-speed cloud generation while retaining 100% offline capability.

- **Library/Class Used:** requests, urllib.request

- **Possible Viva Question: "How does your system handle Groq Cloud API HTTP 429 rate limit errors?"**

- **Ideal Viva Answer:** *"The client catches 429 errors, displays a rate limit warning modal, and automatically falls back to local Ollama or an emergency technical template so execution never crashes."*

Feature 5: Code-First Implementation Studio Engine

- **Purpose:** Synthesizes complete, production-ready multi-file source code packages (80% runnable code, 20% explanation).

- **Location:** [utils/implementation_engine.py](#) -> `execute_implementation_module()` (Line 50)

- **Code Snippet:**

```
smart_tokens = 8192
raw_output = query_fn(system_prompt, user_prompt, agent_role='studio', override_max_tokens=smart_tokens)
```

- **Input:** `project_id`, `active_mode`, `cat_data`, `user_input`.

- **Output:** Dictionary containing multi-file code implementation and execution latency.

- **Step-by-Step Logic:**

1. Loads blueprint context from SQLite `agent_runs`.
2. Calculates an 8,192 token budget.
3. Dispatches query via `agent_role='studio'` using the `imp studio` code Groq key.
4. Returns multi-file code blocks for UI rendering.

- **Why Needed:** Delivers full, un-truncated implementation files instead of high-level bullet summaries.

- **Library/Class Used:** `utils.implementation_engine`

- **Possible Viva Question:** "How do you prevent source code truncation during long module generation?"

- **Ideal Viva Answer:** "We request an extended token budget (`override_max_tokens=8192`) and enforce strict prompt instructions demanding full implementation without placeholders."

Feature 6: Interactive Visual Architecture & Mermaid Pan/Zoom Engine

- **Purpose:** Renders system topology, ERD, and sequence charts in a sandboxed HTML iframe with mouse scroll wheel zoom and click-drag panning.

- **Location:** [pages/projects.py -> render_mermaid_in_streamlit\(\)](#) (Line 280)

- **Code Snippet:**

```
st.iframe('data:text/html;charset=utf-8,' + urllib.parse.quote(html_code), height=height)
```

- **Input:** Mermaid.js markup string, theme choice, height.

- **Output:** Interactive pan/zoom SVG canvas inside iframe.

- **Step-by-Step Logic:**

1. Wraps Mermaid markup in a data-URI HTML document.
2. Injects JS event listeners for mouse scroll wheel zoom and drag pan.
3. Renders top control toolbar for Zoom In, Zoom Out, Reset, and PNG/SVG export.
4. Embeds iframe in Streamlit page.

- **Why Needed:** Allows users to inspect large, detailed system architecture diagrams smoothly without breaking Streamlit page layout.

- **Library/Class Used:** Mermaid.js v10, HTML5 Canvas API

- **Possible Viva Question:** "Why render Mermaid.js inside an isolated Data-URI iframe instead of standard components?"

- **Ideal Viva Answer:** "Data-URI iframes isolate client-side D3/SVG manipulation from Streamlit's Python event loop, allowing smooth 60fps pan/zoom without causing Python script reruns."